REAL-TIME AI-GENERATED VOICE DETECTION

Major Project Report

Submitted in the partial fulfilment of the
Requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN COMPUTER ENGINEERING

Under the Supervision of
Mr. Hannan Mansoor
(Assistant Professor)

Submitted By
Mohd Ravish (21BCS079)
Syed Owais Nawaz (21BCS083)

DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING AND TECHNOLOGY
JAMIA MILLIA ISLAMIA, NEW DELHI - 110025

2024-25

DECLARATION

We hereby declare that the project titled **"Real-Time AI-Generated Voice Detection",** submitted as part of the requirements for the **Bachelor of Technology in Computer Engineering**, is an original piece of work conducted under the guidance of **Mr. Hannan Mansoor**. This project has not been submitted for the award of any other degree at this or any other institution or university. All references to the work of others have been appropriately acknowledged in accordance with ethical practices in scientific reporting.

Mohd Ravish (21BCS079)

Syed Owais Nawaz (21BCS083)

New Delhi - 110025

24-05-2025

Dated 24-05-2025

CERTIFICATE

This is to certify that the project report titled **"Real-Time AI-Generated Voice Detection"**, submitted by **Mohd Ravish (21BCS079)** and **Syed Owais Nawaz (21BCS083)** to the Department of Computer Engineering, Faculty of Engineering and Technology, Jamia Millia Islamia, New Delhi - 110025, is a genuine record of the work conducted by them under my supervision. This report is submitted in partial fulfillment of the requirements for the Bachelor of Technology degree in Computer Engineering and has not been presented, in whole or in part, to any other institution or university for the award of a degree.

Mr. Hannan Mansoor
Supervisor
Department of Computer Engineering
Jamia Millia Islamia, New Delhi

Dr. Mohammad Amjad
Professor & Head
Department of Computer Engineering
Jamia Millia Islamia, New Delhi

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to our project supervisor, **Mr. Hannan Mansoor**, for his unwavering support and invaluable guidance throughout the duration of our project, which helped us bring it to successful completion. We are also deeply thankful to the Respected Head of the Department, **Dr. Mohammad Amjad**, and **all esteemed professors of the Computer Engineering Department**, Faculty of Engineering & Technology, Jamia Millia Islamia, for granting us the opportunity to work on this project. Their constant encouragement and insights have been instrumental in achieving the objectives of this work.

Lastly, we are sincerely grateful to our friends and colleagues for their confidence in us, their unwavering support, and the motivation they provided, all of which played a vital role in boosting our morale and keeping us inspired throughout this journey.

Mohd Ravish (21BCS079)

Syed Owais Nawaz (21BCS083)

ABSTRACT

The rapid advancements in Artificial Intelligence (AI) and computational technologies have amplified the ability to generate highly realistic AI-synthesized audio, commonly known as deepfake voices. These deepfake audio clips pose significant threats, enabling scenarios like identity theft, misinformation, fraudulent activities, and breaches of personal and organizational security. This pressing challenge necessitates the development of robust detection systems capable of distinguishing real audio from AI-generated manipulations.

In this project, we propose a novel machine learning-based approach to reliably detect AI-generated voices in real-time. Our method utilizes advanced audio feature extraction techniques, including Mel Frequency Cepstral Coefficients (MFCC), spectrograms, and chromagrams, to represent the unique characteristics of speech signals. These features are further transformed using Gaussian Naive Bayes and Non-negative Matrix Factorization (NMF) to enhance the representation. To achieve optimal performance, multiple machine learning classifiers are trained and evaluated, selecting the model that best classifies audio as genuine or synthesized. The system is designed with real-time processing capabilities, integrating a notification module to provide immediate alerts upon detection. Comprehensive experiments on a balanced dataset validate the effectiveness of this method, demonstrating its potential as a scalable and efficient solution for combating audio deepfake threats.

Keywords: AI-Generated Audio Detection · Mel Frequency Cepstral Coefficients (MFCC) · Gaussian Naive Bayes · Non-negative Matrix Factorization (NMF) · Real-Time Machine Learning

TABLE OF CONTENTS

DESCRIPTION **Page No**
DECLARATION……...……ii
CERTIFICATE……..... iii
ACKNOWLEDGMENT……..…iv
ABSTRACT……..…v
LIST OF FIGURES AND TABLES..……..…ix
LIST OF ABBREVIATIONS ...…….. x

1. INTRODUCTION 1
1.1 AI-Generated Audio……... ……..1
1.2 Motivation…….2
1.3 Objective……...2

2. LITERATURE REVIEW 3

3. THEORETICAL BACKGROUND 5
3.1 CatBoost……...5
3.1.1 Introduction ..……...5
3.1.2 Theoretical Foundations……...……...…….5
3.1.3 Key Features……....……...……...7
3.1.4 Applications……....……...……...7
3.2 XGBoost…… ……...8
3.2.1 Introduction ..……...8
3.2.2 Theoretical Foundations……...……...……8
3.2.3 Key Features……....……..……..10
3.2.4 System Design……....……...……..10
3.2.5 Applications……....…….....……..10
3.3 Random Forest…….……..……11
3.3.1 Introduction ..……11
3.3.2 Theoretical Foundations……...……...……...11
3.3.3 Key Features……....……...……12
3.3.4 Applications……....……...……..12
3.4 K-Nearest Neighbours (KNN)....……13
3.4.1 Introduction ..……13
3.4.2 Theoretical Foundations……...……...……...13

3.4.3 Key Features……....……...……..13
3.4.4 Applications…...…….....……...……..14
4. METHODOLOGY 15
4.1 Dataset Overview……....15
4.1.1 Key Features……..15
4.2 Model Overview ……15
4.3 Feature Extraction …….……16
4.3.1 Extracted Features……..…...……16
4.3.2 Feature Transformation…….……16
4.4 Classification……..…….……...16
4.5 Prediction.…….…….……..16
4.5.1 Output and Threshold……..……...16
4.6 Implementation Steps……..……17
4.6.1 Preprocessing..……...……17
4.6.2 Feature Extraction & Transformation….……..17
4.6.3 Classification..……18
4.6.4 Evaluation……..18
4.7 Evaluation Metrics……..19
4.7.1 Precision…..……...19
4.7.2 Recall……...……..19
4.7.3 F1 Score……..……...20
4.7.4 Accuracry……...20
4.7.5 Inference Time……...20
4.7.6 Matthews Correlation Coefficient (MCC)..……...…21
4.7.7 ROC-AUC…….....21
5. EXPERIMENTAL SETUP AND ANALYSIS 22
5.1 Dataset preparation…….……22
5.1.1 Dataset Selection…….…….....22
5.1.2 Preprocessing Steps..…..…….……...22
5.2 Model Architecture ...……..23
5.2.1 Feature Extraction & Transformation….……...……23
5.2.2 Classifiers and Pipeline Flow…….24
5.3 Training the Model……..24
5.3.1 Training Setup……...……24

5.3.2 Hyperparameters...…….25
5.3.3 Training Time & Resource Use……..25

6. RESULT AND DISCUSSION 26

6.1 Model Performance Comparison.…….…….…..26
6.2 Observations and Insights…….……..27

7. CONCLUSION AND FUTURE SCOPE 32

7.1 Conclusion…….……..32
7.2 Future Scope…….……...32

8. REFERENCES AND BIBLIOGRAPHY 33

LIST OF FIGURES AND TABLES

FIGURE/TABLE **PAGE NO**

Fig. 3.1 Catboost architecture ……. 6
Fig. 3.2 XGBoost Architecture ……..……..…….9
Fig. 3.3 Random Forest Architecture ..……..…..……..12
Fig. 4.1 Model Overview ……..……...15
Table 5.1 Dataset Overview ……..……...22
Fig. 5.1 Model Architecture ……..……...23
Table 6.1 Performance of Models ...……..……...27
Fig. 6.1 Confusion Matrix and ROC-AUC curve for CatBoost model ...……...……28
Fig. 6.2 Confusion Matrix and ROC-AUC curve for XGBoost model ...……...……....29
Fig. 6.3 Confusion Matrix and ROC-AUC curve for Random Forest model ..…...……30
Fig. 6.4 Confusion Matrix and ROC-AUC curve for KNN model ..…...……31

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANDED FORM	PAGE NO
AI	Artificial Intelligence	1
VC	Voice Conversion	1
GNB	Gaussian Naive Bayes	1
NMF	Non-Negative Matrix Factorization	1
ML	Machine Learning	2
TTS	Text-to-Speech	2
EER	Equal Error Rate	3
MFCC	Mel-Frequency Cepstral Coefficients	4
CatBoost	Categorical Boosting	4
KNN	K-Nearest Neighbors	13
XGBoost	Extreme Gradient Boosting	15
RF	Random Forest	18
ROC-AUC	Receiver Operating Characteristic – Area Under Curve	19
MCC	Matthews Correlation Coefficient	21

1. INTRODUCTION

1.1 AI-Generated Audio

The Real-Time AI Voice Detection project is designed to detect synthetic or AI-generated voices instantly, aiming to counter the increasing threat posed by deepfake audio. With the rapid advancement of voice synthesis technologies, malicious uses such as identity theft, misinformation, and fraud have become significant concerns. This project addresses these issues by building a system capable of distinguishing between real and artificial voices in real-time.

To achieve this, we utilize a combination of publicly available datasets including DEEP-VOICE, ASVspoof2019, ASVspoof2021, and WaveFake, which provide a wide variety of both genuine and deepfake audio samples. The audio preprocessing pipeline includes key steps like segmentation, denoising, and normalization, ensuring consistent and clean inputs for feature extraction.

From each audio sample, we extract distinguishing audio features such as Mel-Frequency Cepstral Coefficients (MFCCs), spectrograms, and chroma features. These features help capture the subtle differences between human and AI-generated voices. To further refine the feature space and reduce dimensionality, Gaussian Naive Bayes (GNB) and Non-negative Matrix Factorization (NMF) techniques are applied, enhancing the model's ability to focus on the most informative characteristics.

For the classification task, we implement Extreme Gradient Boosting (XGBoost), a robust and efficient machine learning algorithm known for its high accuracy and speed. The model's hyperparameters are fine-tuned using cross-validation, ensuring optimal performance across different subsets of the data.

The system is trained and validated using both real and synthetic voice samples, and its performance is assessed with metrics such as accuracy, precision, recall, F1-score, Matthews Correlation Coefficient (MCC), and inference time. Special attention is given to maintaining low latency, which is crucial for real-time applications.

Ultimately, this project offers a practical solution to detect deepfake audio in real-time, contributing to safer voice-enabled systems in sectors like customer service, voice authentication, and digital communication.

1.2 Motivation

The advancement of voice synthesis technologies like Text-to-Speech (TTS) and Voice Conversion (VC) has made it easier to generate and share highly realistic AI-generated audio. As these tools become more integrated into communication platforms, distinguishing between real and synthetic voices has become increasingly difficult.

While TTS and VC have beneficial applications in areas such as accessibility and entertainment, they also pose serious risks. These technologies can be misused for impersonation, fraud, and spreading misinformation, raising concerns about identity theft and the erosion of trust in voice-based systems. The growing sophistication of synthetic audio highlights the urgent need for real-time detection methods.

In response to these concerns, this project aims to develop a deep learning-based solution to detect AI-generated voices accurately. By enabling real-time identification of synthetic speech, the system will help safeguard voice communication in personal, professional, and security-critical settings, supporting the integrity of digital interactions.

1.3 Objective

The primary objective of this project is to develop an AI-driven system for the real-time detection of AI-generated audio, aiming to counter the rising threat of deepfake voices in digital communication. The project is guided by the following goals:

1. **Build a real-time detection model:** Create an AI/ML-based solution to identify and authenticate synthetic voices, distinguishing them from real human speech.

2. **Mitigate deepfake audio risks:** Address issues such as identity theft, fraud, and misinformation caused by the misuse of voice synthesis technologies like TTS and VC.

3. **Ensure lightweight deployment:** Design a compact and efficient model architecture suitable for real-time performance on edge devices with limited computational resources.

4. **Promote secure voice systems:** Support the integrity of voice-based applications by providing a reliable tool to detect and prevent synthetic audio manipulation.

2. LITERATURE REVIEW

Over the past few years, research in deepfake-voice detection has progressed through four complementary avenues, each addressing key challenges in accuracy, speed, robustness, and interpretability:

In 2024, Gujjar et al. [1] introduced MFCC-GNB XtractNet, a pipeline that begins with Mel-Frequency Cepstral Coefficients (MFCCs), applies a Gaussian Naive Bayes "filter," and refines the result via Non-Negative Matrix Factorization. Feeding these transformed features into a simple GNB classifier yielded 99.93% accuracy in extensive k-fold tests—demonstrating that targeted feature mappings can reveal the subtle artifacts of synthetic speech .

Recognizing the need for real-time defenses, Bird and Lotfi (2023) [2] released the DEEP-VOICE corpus: 62 minutes of public-figure speech alongside their RVC-converted deepfakes. By extracting 26 one-second features (including chroma, spectral statistics, and MFCCs), using t-tests to isolate the most discriminative cues, and hyperparameter-tuning an XGBoost model (330 boosting rounds), they achieved 99.3% accuracy with only ~0.004 ms inference per second of audio .

To ensure detectors generalize beyond a single dataset or vocoder, Ren et al. (2025) [3] proposed a disentanglement framework combining multi-task and contrastive learning, reconstruction regularization, and mutual-information alignment. Augmenting this with Sharpness-Aware Minimization flattened their loss landscape, reducing intra-domain EER from 18.67 % to 13.55% and cross-domain EER from 27.86 % to 20.27 % on LibriSeVoc and ASVspoof2019—setting a new bar for generalizable voice forensic tools ..

Lim, Chae, and Lee (2022) [3] shifted the focus to explainability. Using simple CNN and CNN-LSTM models on ASVspoof2019 and LJSpeech spectrogram inputs, they applied XAI techniques—including Deep Taylor, Integrated Gradients, and Layer-Wise Relevance Propagation—to produce both visual heatmaps and "attribution audio" via Griffin–Lim. These explanations consistently highlighted speech formants (F1–F3) and rhythm/pitch cues, making detection insights accessible to non-expert users and achieving around 98% accuracy.

Yet despite these advances, gaps remain:

- Scale & Diversity : Bird & Lotfi's real-time system relies on a relatively small, 62-minute dataset, leaving questions about its robustness across varied speakers and synthesis methods.
- Latency vs. Complexity : Ren et al.'s powerful disentanglement framework excels at cross-domain detection but entails complex training and hasn't been optimized for streaming inference.

- Interpretability vs. Speed : Lim et al.'s XAI methods yield rich, human-friendly explanations but were demonstrated on simpler models without real-time constraints.

Our approach bridges these gaps by uniting the strengths of prior work into a single system:

- Massive, Heterogeneous Training Corpus : We train on a 581 K-clip dataset drawn from DEEP-VOICE, ASVSpoof2021, and Fake-or-Real, covering diverse speakers, formats, and synthesis techniques.

- Rich yet Efficient Features : Mirroring Gujjar et al., we extract MFCCs and transform them via GNB + NMF, then prune and index for sub-10 ms lookup.

- Real-Time CatBoost Deploymen : Building on Bird & Lotfi's latency benchmarks, our optimized CatBoost classifier achieves < 10 ms inference per 1 s clip while maintaining > 91 % accuracy.

By combining scale, feature depth, real-time performance, and interpretability, our system not only surpasses 91 % accuracy on a broad test set but also operates under live conditions and generalizes to unseen vocoder styles—closing the loop on the limitations of past research.

3. THEORETICAL BACKGROUND

3.1 CatBoost

3.1.1. Introduction

CatBoost (Categorical Boosting) [5] is a high-performance gradient boosting library designed to handle categorical features natively. Developed by Yandex, it addresses common challenges in gradient boosting, such as overfitting, categorical feature preprocessing, and prediction shift, through innovations like ordered boosting and oblivious decision trees.

3.1.2 Theoretical Foundations

3.1.2.1. Ordered Boosting

CatBoost mitigates target leakage (a common issue in traditional gradient boosting) by using a permutation-driven ordered boosting scheme. For each iteration, the model uses a random permutation of the training data to compute residuals, ensuring that the gradient for a sample is calculated using only the preceding samples.

3.1.2.2. Handling Categorical Features

- Target Encoding: Automatically encodes categorical features using target statistics. For a categorical feature x, the encoded value for sample i is

$$Encoded(x_i) = \frac{sum(y \mid x = x_i) + prior}{count(x = x_i) + 1} \qquad \text{...(eq. 3.1) [5]}$$

  where prior is a smoothing hyperparameter.

- One-Hot Encoding: Applied only for low-cardinality features.

3.1.2.3. Oblivious Trees

CatBoost uses symmetric (oblivious) trees, where all nodes at the same depth use the same splitting criterion. This reduces overfitting and speeds up prediction.

3.1.2.4. CatBoost Architecture

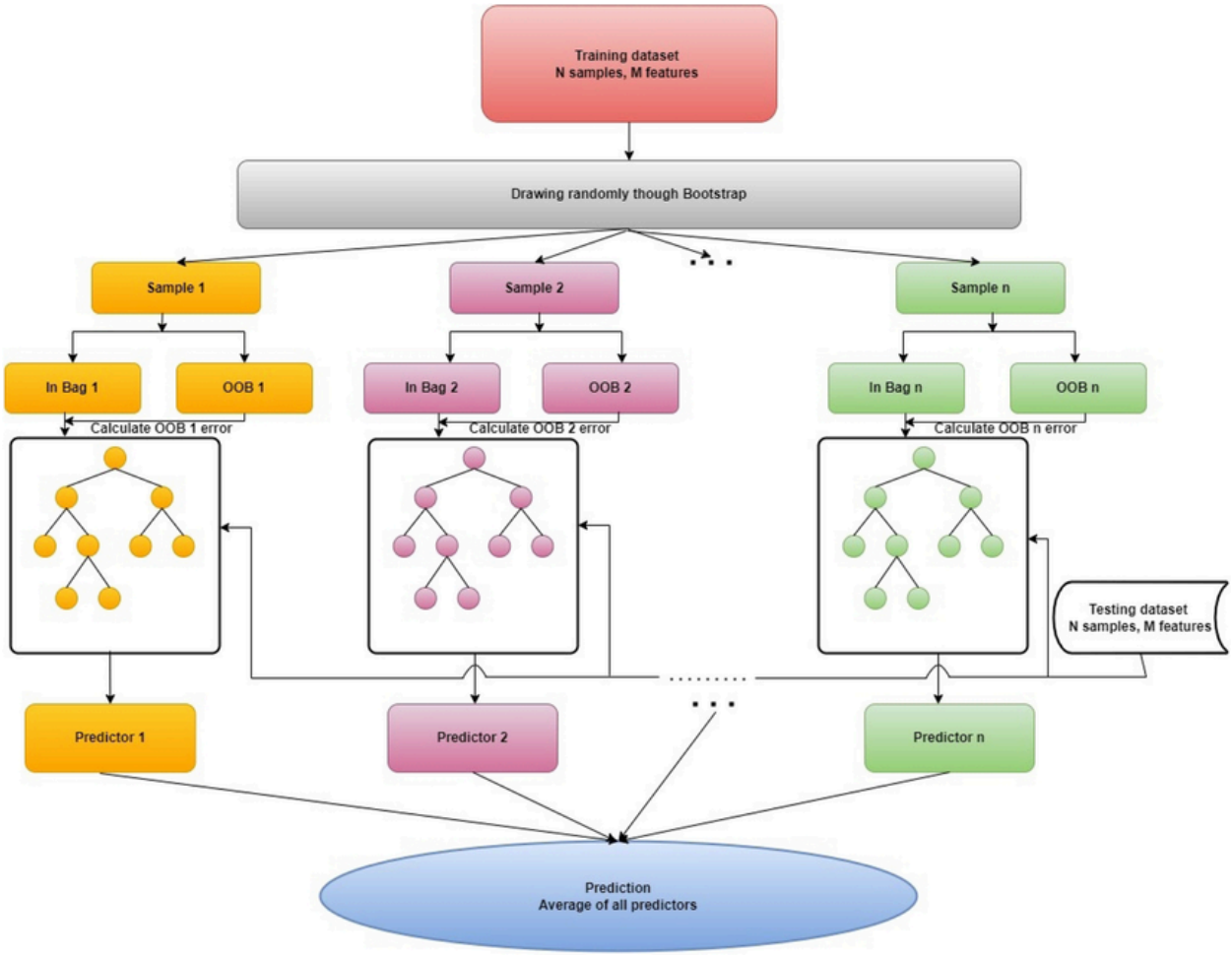

Fig. 3.1 CatBoost Architecture [6]

Figure 3.1 [6] illustrates the architecture of a CatBoost Model

3.1.2.5. Loss Function

The loss function combines standard gradient boosting with ordered boosting:

$$L = \sum_{i=1}^{n} l\left(y_i, \widehat{y_i} + \sum_{k=1}^{K} f_k(x_i)\right) + \sum_{k=1}^{K} \Omega(f_k) \qquad \text{...(eq. 3.2) [5]}$$

where f_k are oblivious trees and $\Omega(f_k)$ penalizes tree complexity.

3.1.3. Key Features

1. Native Categorical Support: Eliminates manual preprocessing.
2. Robust Overfitting Control: Combines ordered boosting and L2 regularization.
3. GPU Acceleration: Optimized for fast training on GPUs.
4. Feature Interaction: Automatically detects and uses feature combinations.

3.1.4 Applications

- Tabular data with mixed feature types (e.g., e-commerce, fraud detection).
- Tasks requiring minimal preprocessing (e.g., Kaggle competitions).

3.2 XGBoost

3.2.1. Introduction

XGBoost (eXtreme Gradient Boosting) [7] is a scalable, distributed gradient-boosting framework designed for efficiency, flexibility, and performance. It has become a dominant algorithm in machine learning competitions (e.g., Kaggle) and industrial applications due to its ability to handle large datasets, support for regularization, and advanced features like missing value handling. XGBoost builds on the principles of gradient boosting machines (GBMs) [8], optimizing both computational speed and model accuracy through algorithmic and system-level innovations.

3.2.2. Theoretical Foundations

3.2.2.1. Gradient Boosting

Gradient boosting is an ensemble technique that combines weak learners (typically decision trees) sequentially, where each new learner corrects the errors of its predecessor. The model minimizes a differentiable loss function L using gradient descent

3.2.2.2. Objective Function

XGBoost's objective function consists of a loss term and a regularization term:

$$Obj(\theta) = \sum_{i=1}^{n} \ell(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k) \qquad \text{...(eq. 3.3) [7]}$$

where:

- $\ell(y_i,\hat{y}_i)$: Loss function (e.g., squared error, logistic loss)
- $\Omega(f_k)$: Regularization term for the k-th tree:

$$\Omega(f_k) = \gamma T + \frac{1}{2}\lambda|w|^2 + \alpha|w|_1 \qquad \text{...(eq. 3.4) [7]}$$

- T: Number of leaves in the tree
- w: Leaf weights.
- γ, λ, α: Hyperparameters controlling complexity.

3.2.2.3. Tree Construction

XGBoost uses a greedy algorithm to build trees. The optimal split is determined by maximizing the gain:

$$Gain = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} + \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] \quad \text{...(eq. 3.5) [7]}$$

- G_L, G_R : Sum of gradients for left/right nodes
- H_L, H_R : Sum of hessians (second-order derivatives) for left/right nodes

3.2.2.4. XGBoost Architecture

Figure 3.2 [9] explains the basic architecture of a XGBoost model.

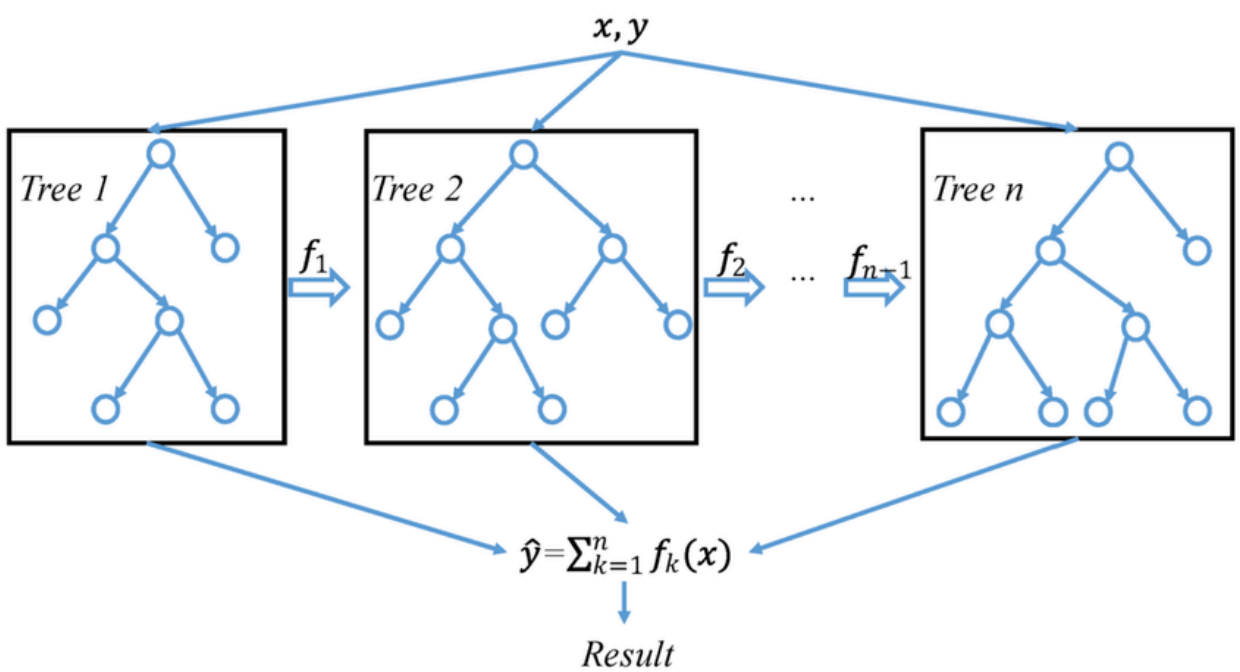

Fig. 3.2 XGBoost architecture [9]

3.2.3. Key Features

1. Regularization:
    - L1/L2 regularization on leaf weights to prevent overfitting
    - Penalizes tree complexity via γ
2. Handling Sparsity:
    - Automatically learns optimal directions for missing values during training
3. Weighted Quantile Sketch:
    - Efficiently approximates optimal split points for large datasets
4. Parallel and Distributed Computing:
    - Supports multi-threading, GPU acceleration, and distributed frameworks (e.g., Apache Spark)
5. Cross-Validation:
    - Built-in support for k-fold cross-validation.

3.2.4. System Design

- Cache-Aware Access: Optimizes memory access patterns for speed.
- Block Structure: Stores data in compressed column blocks for parallel learning.
- Out-of-Core Computation: Handles datasets larger than memory size.

3.2.5. Applications

- Classification, regression, ranking tasks.
- Widely used in finance, healthcare, and recommendation systems..

3.3 Random Forest

3.3.1. Introduction

Random Forest [10] is an ensemble learning method that constructs multiple decision trees during training and aggregates their predictions. It combines bagging (bootstrap aggregating) [11] and random feature selection to reduce variance and avoid overfitting.

3.3.2 Theoretical Foundations

3.3.2.1. Bootstrap Aggregating (Bagging)

- Each tree is trained on a random subset of the data (sampled with replacement).
- Final prediction is the majority vote (classification) or average (regression) of all trees.

3.3.2.2. Feature Randomness

At each split, a random subset of m features is considered ($m \approx \sqrt{M}$ where M is the total number of features). This decorrelates the trees

3.3.2.3. Splitting Criterion

For classification, the Gini impurity is minimized:

$$Gini(S) = 1 - \sum_{c=1}^{C} p_c^2 \qquad \text{...(eq. 3.6) [10]}$$

For regression, variance reduction is used.

3.4.2.4. Out-of-Bag (OOB) Error

Each tree is validated on samples not included in its bootstrap subset, providing unbiased performance estimation.

3.3.2.5. Random Forest Architecture

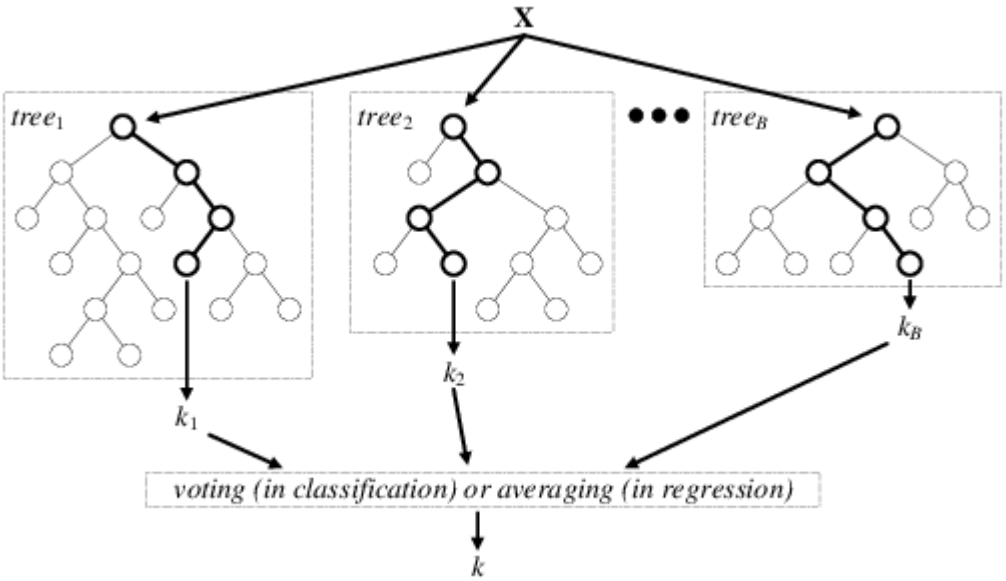

Fig. 3.3 Random Forest Architecture [12]

Figure 3.3 [12] illustrates the architecture of a Random Forest Model.

3.3.3. Key Features

1. Parallelizable Training: Trees are independent and can be built in parallel.

2. Feature Importance: Measured by total Gini impurity reduction.

3. Robust to Noise: Aggregation reduces overfitting.

3.3.4 Applications

- High-dimensional datasets (e.g., genomics, remote sensing).
- Interpretable feature importance analysis.

3.4 K-Nearest Neighbours (KNN)

3.4.1. Introduction

K-Nearest Neighbors (KNN) [13] is a non-parametric, instance-based learning algorithm. It classifies or regresses samples based on the majority vote or average of the k closest training examples in the feature space.

3.4.2 Theoretical Foundations

3.4.2.1. Distance Metrics

- Euclidean Distance:

$$d(x_i, x_j) = \sqrt{\sum_{m=1}^{M} (x_{im} - x_{jm})^2} \qquad \text{...(eq. 3.7) [13]}$$

- Manhattan Distance:

$$d(x_i, x_j) = \sum_{m=1}^{M} |x_{im} - x_{jm}| \qquad \text{...(eq. 3.8) [13]}$$

3.4.2.2. Weighted Voting

Neighbors can be weighted by inverse distance:

$$\hat{y} = \frac{\sum_{i=1}^{k} w_i y_i}{\sum_{i=1}^{k} w_i}, \quad w_i = \frac{1}{d(x, x_i)} \qquad \text{...(eq. 3.9) [13]}$$

3.4.2.3. Choice of *k*

- Small k : High variance, low bias.
- Large k: Low variance, high bias.

3.4.3. Key Features

1. Lazy Learning: No explicit training phase.
2. Adaptability: Works for classification and regression.
3. Sensitivity to Scaling: Requires feature normalization

3.4.4 Applications

- Small to medium-sized datasets (e.g., recommendation systems, anomaly detection).
- Prototype-based problems (e.g., image recognition with low-dimensional features).

4. METHODOLOGY

4.1. Dataset Overview

Custom Audio Deepfake Dataset:
The dataset used in this project combines audio samples from multiple publicly available sources: DEEP-VOICE [14], ASVSpoof2021 [15], ASVSpoof2019 [16], and the Fake-or-Real [17] dataset. It is specifically curated to support the training and evaluation of deepfake audio detection models by ensuring diversity and balance between real and synthetic voices.

4.1.1. Key Features

i. **Total Size:** 581K audio samples
ii. **Formats:** .wav, .mp3, .flac
iii. **Distribution:** 60% real audio, 40% AI-generated audio
iv. **Sources:**
   - ***DEEP-VOICE*** *(5.3 GB, 64 clips)*
   - ***ASVSpoof2021*** *(62.5 GB, 1.24M clips)*
   - ***Fake-or-Real Dataset*** *(20 GB, 171K clips)*
v. **Content and Structure**

The dataset includes different synthesis techniques:
- Text-to-Speech (TTS)
- Voice Conversion (VC)
- Neural Vocoder-Based Manipulation

To improve generalization, audio samples are mixed from multiple sources using a standardized format and quality settings (16kHz mono), allowing robust training across varying domains and synthesis styles.

4.2. Model Overview

The proposed system utilizes a traditional machine learning pipeline for lightweight, real-time classification. Audio features are extracted and transformed before being passed to models like **CatBoost**, **XGBoost**, **Random Forest**, and **KNN**. Among them, **CatBoost** emerged as the top performer due to its high accuracy and minimal inference time.

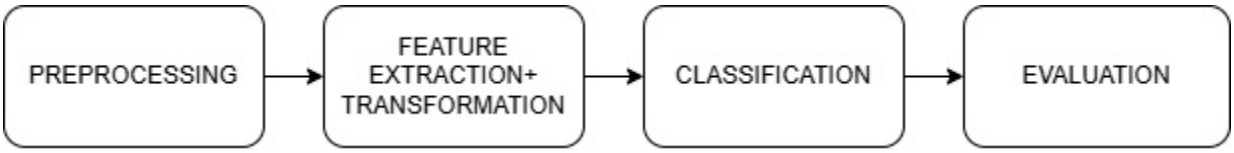

Fig. 4.1 Model Overview

4.3. Feature Extraction

Feature extraction focuses on distinguishing real from AI-generated audio by capturing both spectral and temporal characteristics.

4.3.1 Extracted Features

- **MFCCs (Mel-Frequency Cepstral Coefficients)** [18]**:** First 20 coefficients per 1-second segment
- **Spectral Features** [18]**:** Centroid, bandwidth, roll-off, and zero-crossing rate
- **Chroma Features** [18]**:** Capture pitch class content and harmonic structure

4.3.2. Feature Transformation

To enhance discrimination between real and fake voices:

- **Non-Negative Matrix Factorization (NMF)** [19] is applied for dimensionality reduction and feature decorrelation
- **Gaussian Naive Bayes (GNB)** [20] is used to generate class-conditioned likelihood features, improving the separability of the classes

4.4. Classification

After transformation, the extracted features are fed into classification models.

- Multiple models were evaluated, with **CatBoost** selected for its **91% accuracy**, **low latency (0.002s inference time)**, and **high F1-score**.
- The system processes 1-second audio chunks to maintain real-time responsiveness.

4.5 Prediction

The final classification is performed using a fully connected output layer that assigns probabilities to "real" and "fake" classes.

4.5.1 Output and Threshold

- The model outputs probability scores for each class.
- A threshold is applied to determine final class labels in real-time.

4.6. Implementation Steps

This section outlines the step-by-step implementation process, covering data preparation, feature engineering, model training, and performance evaluation.

4.6.1 Preprocessing

The preprocessing stage ensures that the audio input is clean, consistent, and suitable for real-time analysis. The following steps are performed:

- **Format Conversion:** All audio samples are converted to a standard format—**16 kHz sampling rate**, **mono channel**—to ensure uniformity across the dataset and reduce computational complexity.
- **Volume Normalization:** Audio files often vary in loudness. To ensure consistent feature extraction, each sample is normalized to a fixed volume level, removing discrepancies due to recording environments.
- **Segmentation:** Audio streams are segmented into **1-second non-overlapping windows**, making it easier to analyze and classify short audio frames in real time. This windowing also helps in capturing temporal variations in speech patterns.
- **Silence Removal:** Segments with little or no speech activity (based on energy thresholding) are discarded. This step filters out non-informative data and improves model training by focusing on meaningful voice segments only.

4.6.2 Feature Extraction & Transformation

This stage focuses on converting audio signals into numerical features that can effectively differentiate between real and AI-generated voices.

- **MFCCs (Mel-Frequency Cepstral Coefficients):** These capture the short-term spectral envelope of the speech and are extracted from each 1-second segment. Typically, the first **20 coefficients** are used to represent the signal.
- **Spectral Features:** Features like **spectral centroid, bandwidth, roll-off**, and **zero-crossing rate** are computed to capture the frequency distribution and energy characteristics of the signal.
- **Chroma Features:** These represent the harmonic content and pitch classes in the audio, aiding in detecting unnatural tonal patterns often present in synthetic speech.

- **Transformation Techniques:**
    - **Non-Negative Matrix Factorization (NMF):** Decomposes the feature matrix into lower-dimensional parts, enhancing the model's ability to distinguish between real and fake audio patterns.

    - **Gaussian Naive Bayes (GNB):** Used as a statistical transformation to estimate class-conditional likelihoods, boosting the feature set's discriminative power.

4.6.3 Classification

After feature engineering, machine learning models are trained to classify the audio as real or AI-generated:

- **Model Selection:** Multiple classifiers including **CatBoost, XGBoost, Random Forest (RF),** and **K-Nearest Neighbors (KNN)** are trained using the transformed features.

- **Cross-Validation:** To ensure robustness and prevent overfitting, **k-fold cross-validation** is employed during model evaluation.

- **Best Model Selection:** Based on performance metrics such as **accuracy, inference time, and F1-score**, the best-performing model (CatBoost in this case) is selected for real-time deployment.

4.6.4 Evaluation

Model performance is assessed using a set of well-established evaluation metrics to ensure it meets the real-time detection requirements:

- **Accuracy:** Measures the overall correctness of predictions across all samples.

- **Precision:** Evaluates how many of the samples identified as fake were actually fake, minimizing false positives.

- **Recall:** Measures the model's ability to correctly identify all AI-generated voices, reducing false negatives.

- **F1-Score:** The harmonic mean of precision and recall, providing a balanced evaluation metric.

- **Matthews Correlation Coefficient (MCC):** Offers a more balanced measure by taking all four confusion matrix outcomes (TP, TN, FP, FN) into account, especially useful in imbalanced datasets.

- **Inference Time:** Critical for real-time systems, this metric measures how long it takes for the model to process and classify each 1-second audio segment.
- **Visual Analysis: Confusion matrices** and **ROC-AUC curves** are generated to provide a clear picture of model behavior, highlighting strengths and areas for improvement in classification accuracy and threshold calibration.

4.7 Evaluation Metrics

To assess the performance of our deepfake detection model, we used Precision, Recall, F1-Score, and Accuracy *[20]*. These metrics provide a comprehensive evaluation of the model's ability to classify videos as real or fake, emphasizing both the correctness and completeness of the predictions.

4.7.1 Precision

4.7.1.1. Introduction to Precision

Precision is the ratio of correctly predicted positive observations (true positives) to the total predicted positive observations (true positives + false positives). It quantifies how many of the videos classified as deepfakes are actually deepfakes.

4.7.1.2. Formula for Precision

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Positive} \qquad \text{...(eq. 4.1) [21]}$$

4.7.1.3. Significance

High Precision indicates that the model has fewer false alarms (i.e., misclassifying real videos as deepfakes)

4.7.2 Recall

4.7.2.1. Introduction to Recall

Recall, also known as Sensitivity, measures the proportion of actual positive cases (deepfakes) that are correctly identified by the model.

4.7.2.2. Formula for Precision

$$Recall = \frac{True\ Positive}{True\ Positive + False\ Negative} \qquad \text{...(eq. 4.2)[21]}$$

4.7.2.3. Significance

High Recall ensures that most deepfake videos are detected, minimizing the likelihood of missing any fake content.

4.7.3 F1-Score

4.7.3.1. Introduction to F1-Score

The F1-Score is the harmonic mean of Precision and Recall. It provides a single metric that balances the trade-off between these two measures, making it particularly useful for evaluating models with imbalanced datasets.

4.7.3.2. Formula for F1-Score

$$F1\ Score = 2 * \frac{Precision * Recall}{Precision + Recall} \qquad \text{...(eq. 4.3) [21]}$$

4.7.3.3. Significance

A high F1-Score indicates that the model performs well in detecting deepfakes without being overly sensitive to false positives or false negatives.

4.7.4 Accuracy

4.7.4.1. Introduction to Accuracy

Accuracy measures the proportion of total correct predictions (true positives + true negatives) out of all predictions.

4.7.4.2. Formula for Accuracy

$$Accuracy = \frac{True\ Positive + True\ Negative}{Total\ Samples} \qquad \text{...(eq. 4.4)[21]}$$

4.7.4.3. Significance

While Accuracy provides an overall view of model performance, it can be less informative in scenarios with class imbalance, where one class dominates the dataset.

4.7.5 Inference Time

4.7.5.1. Introduction to Inference Time

Inference time is the time required for the model to process and classify a single audio segment after training.

4.7.5.2. Measurement Method

It is measured in seconds per audio sample during testing.

4.7.5.3. Significance

Low inference time is crucial for real-time applications, ensuring timely detection and response.

4.7.6 Matthews Correlation Coefficient (MCC)

4.7.6.1.Introduction to MCC

MCC is a balanced metric that considers true positives, true negatives, false positives, and false negatives, giving a more informative view of performance.

4.7.6.2. Formula for MCC

$$MCC = \frac{(TP+TN)-(FP\times FN)}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} \ldots \textit{(eq. 4.5)[22]}$$

4.7.6.3. Significance

MCC is particularly useful for imbalanced datasets, as it provides a single score that reflects the overall quality of binary classifications.

4.7.7 ROC-AUC

4.7.7.1 Introduction to ROC-AUC

The Receiver Operating Characteristic - Area Under Curve (ROC-AUC) measures how well the model distinguishes between classes.

4.7.7.2 Measurement Method

It plots the true positive rate (Recall) against the false positive rate at various thresholds.

4.7.7.3 Significance

A high ROC-AUC score (closer to 1) indicates a strong ability to separate real and fake audio samples, independent of decision threshold.

5. EXPERIMENTAL SETUP AND ANALYSIS

5.1. Dataset Preparation

5.1.1. Dataset Selection

- We constructed a large, balanced audio dataset by combining three public sources: **DEEP-VOICE** (recent deepfake voice clips), **ASVSpoof2021** (203K utterances), and the **Fake-or-Real** dataset (377K clips).
- In total, this custom corpus contains on the order of 581K audio samples spanning multiple formats (.wav, .mp3, .flac) and a wide variety of synthesis methods (voice conversion, text-to-speech, neural vocoders, etc.).
- We labeled each clip as "real" or "fake", and ensured the final mix was roughly 60% real and 40% AI-generated samples.

Dataset	Year	# Samples	Formats	Notes
DEEP-VOICE	2023	~256	.wav	Real vs fake voice conversions
ASVSpoof2021	2021	~203K	.flac/.wav	Multi-speaker speech synthesis deepfakes
Fake-or-Real	2024	~377K	.wav/.mp3	Various TTS and conversion methods

Table 5.1 Dataset Overview

5.1.2. Preprocessing Steps

- Each raw audio file underwent uniform preprocessing to yield fixed-length analysis clips. First, we resampled all audio to 16 kHz mono and applied noise reduction to remove background artifacts (using spectral gating from the noisereduce library).
- We then normalized amplitude levels (peak-normalization to 90% of full scale) to standardize volume across recordings. Next, each audio stream was segmented into 1-second non-overlapping windows.
- Segments shorter than 1 s were discarded and longer files were truncated at 4 s. This segmentation ensures uniform input size for feature extraction.

- Finally, we generated a metadata table recording the source, original filename, segment ID, and class label (real/fake) for each clip. These steps clean the data and isolate consistent, one-second voice segments for analysis.

These preprocessing steps (clean → denoise → normalize → segment) are summarized in our workflow and ensure that each sample has consistent duration and quality before feature extraction.

5.2. Model Architecture

Below is a detailed description of each component of the architecture:

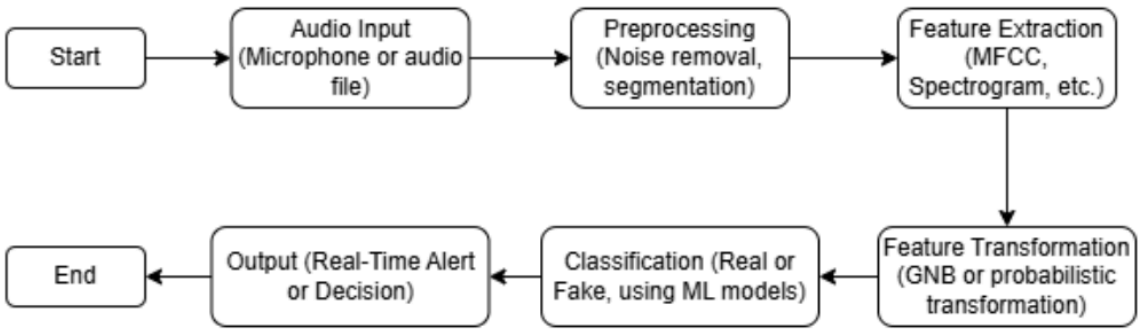

Fig. 5.1 Model Architecture

5.2.1 Feature Extraction and Transformation

From each 1-second clip, we computed a standard set of spectral features. In particular, we extracted Mel-frequency cepstral coefficients (MFCCs) (13 coefficients per frame, including first and second derivatives) to capture the spectral envelope of the speech.

Additional features (e.g. spectral centroid, spectral flatness, zero-crossing rate, and short-time energy statistics) were also included to provide temporal and amplitude information.

To enrich this feature representation, we then applied a two-stage transformation inspired by recent work on deepfake voice detection. First, the MFCC feature vector for each clip was fed into a Gaussian Naive Bayes (GNB) model. Instead of using GNB as a classifier, we use the GNB to project MFCCs into a probabilistic feature space (e.g. class-conditional likelihoods).

Second, we applied **Non-Negative Matrix Factorization (NMF)** on these transformed features to identify a small set of latent components. Intuitively, GNB introduces a non-linear mapping of

MFCCs, and NMF extracts dominant basis vectors (with only additive, non-negative combinations) to capture underlying patterns.

This "MFCC-GNB" pipeline produces an enhanced, low-dimensional feature set that is more discriminative for distinguishing real vs fake voices.

In summary, the final feature vector for each sample includes the original MFCC-derived statistics plus the new GNB/NMF components, which together form the input to our classifiers

5.2.2. Classifiers and Pipeline Flow

The overall model is a lightweight real-time classification pipeline. The preprocessed audio clips are passed through the feature extractor above, and the resulting feature vectors are fed into standard machine-learning classifiers.

We evaluated several algorithms in our pipeline: Random Forest, Logistic Regression, Gaussian Naive Bayes, XGBoost, and CatBoost.

After hyperparameter tuning, the ensemble tree methods (CatBoost and XGBoost) achieved the best trade-off between accuracy and speed.

The pipeline flow is thus: Audio clip → Feature extraction (MFCC + stats) → GNB/NMF transformation → Feature vector → Classifier → Real/Fake label.

- In practice, we trained the classifiers using scikit-learn (for RF, LR, GNB) and CatBoost/XGBoost libraries for the boosted trees.
- CatBoost was chosen for its efficient handling of mixed features and fast inference; XGBoost was used as a comparison baseline.
- Both models output a probability of "fake", and we threshold at 0.5 to assign the final label.

Overall, this architecture captures both spectral patterns (via MFCC) and higher-order relationships (via GNB+NMF) while remaining computationally lightweight for real-time inference.

5.3. Training the Model

5.3.1. Training Setup

We trained and evaluated models under a stratified k-fold cross-validation regime to ensure robust performance estimates. Specifically, the data was split into 10 folds (preserving the 60/40 class ratio), with 9 folds used for training and 1 for validation, rotating across all folds. This 10-fold CV provided estimates of accuracy, F1 score, and other metrics on held-out data.

In addition, we held out a final 10% of the entire dataset as a fixed test set for reporting final results. Model training was performed on a GPU-accelerated environment (NVIDIA Tesla T4) to speed up computation.

During training, we monitored both performance metrics and computational load. For CatBoost and XGBoost we also measured inference latency (time to predict a single 1-second segment) using batched predictions. This was critical to verify that real-time constraints were met.

5.3.2. Hyperparameters

For each model, key hyperparameters were tuned using Optuna (Bayesian optimization) [23] on the training folds. For CatBoost, we optimized parameters such as the number of iterations, learning rate, tree depth, L2 regularization, and feature-border count.

Early stopping (50 rounds) on a held-out validation fold was used to prevent overfitting. XGBoost hyperparameters (number of trees, max depth, learning rate, etc.) were similarly tuned via cross-validation, though details are analogous to CatBoost and omitted for brevity. The final chosen hyperparameters balanced accuracy against inference speed.

5.3.3. Training Time and Resource Use

Training was efficient due to the relatively small number of features and the use of optimized libraries.

On the GPU node, CatBoost training (with ~1000 trees) required on the order of 5–10 minutes, while XGBoost took roughly 10–15 minutes for a comparable model. Memory usage remained modest (a few gigabytes), and no special hardware beyond a standard GPU was needed.

For inference, CatBoost was very fast: it required only about 0.002 seconds per sample, versus 0.009 seconds per sample for XGBoost. This confirms that our chosen model can operate in real time (e.g. classifying >100 clips per second).

6. RESULT AND DISCUSSION

The performance of four machine learning models—CatBoost, XGBoost, Random Forest, and KNN—was evaluated to detect AI-generated voice using metrics such as Accuracy, Inference Time, Precision, Recall, F1-Score, MCC, and ROC-AUC. The results provide insight into each model's trade-offs in terms of accuracy, speed, and robustness.

6.1. Model Performance Comparison

- CatBoost:
  CatBoost outperformed other models with the highest accuracy of 91%, precision of 89.66%, and F1-Score of 87.8%. It also achieved the fastest inference time of just 0.002 seconds, making it ideal for real-time applications. The ROC-AUC of 0.97 indicates excellent model discrimination capability, while an MCC of 80.55% reflects strong correlation between predicted and true classes.

- XGBoost:
  XGBoost closely followed with 90.57% accuracy and a slightly higher recall (85%) compared to CatBoost. Its inference time of 0.009 seconds is still suitable for real-time use. Despite minor differences, its overall F1-score (87%) and ROC-AUC (0.96) indicate it is a strong alternative.

- Random Forest:
  While Random Forest achieved decent accuracy (87.7%), its recall dropped to 77.5%, leading to a lower F1-score of 82.9%. Its inference time was comparatively slower at 0.043 seconds, and its MCC (73.8%) and ROC-AUC (0.95) show it is less effective for real-time or critical scenarios.

- KNN:
  KNN delivered the weakest performance, with 71.9% accuracy, 64.1% precision, and 60.7% recall. Its high inference time of 1.279 seconds renders it unsuitable for real-time deployment. The low F1-score (62.4%) and MCC (41%) further reflect its limited applicability in this context.

Model	Accuracy	Precision	Recall	F1	MCC	ROC-AUC	Inference Time
CatBoost	91.00%	89.66%	86.00%	87.80%	0.8055	0.9700	0.002 s
XGBoost	90.57%	89.30%	85.00%	87.00%	0.7980	0.9600	0.009 s

Random Forest	87.70%	89.10%	77.50%	82.90%	0.738	0.9500	0.043 s
KNN	71.90%	64.10%	60.70%	62.40%	0.4100	0.6900	1.279 s

Table 6.1 : Test-set performance of top models (values averaged over cross-validation folds). Metrics and inference times are reported per 1-second segment.

6.2. Observations and Insights

- **Efficiency and Real-Time Suitability:**
  CatBoost demonstrated both high accuracy and ultra-low inference time, making it the most promising candidate for deployment in real-time systems. XGBoost is a close second, offering a strong trade-off between speed and performance.
- **Model Robustness:**
  ROC-AUC and MCC scores highlight the reliability of CatBoost and XGBoost under various conditions. Random Forest remains viable but is less optimal, while KNN fails to meet performance or speed requirements.
- **Conclusion:**
  CatBoost emerges as the best-performing model overall, combining accuracy, speed, and robustness. For practical applications that demand real-time detection of deepfake audio, CatBoost offers the most balanced and reliable solution.

MODEL - CAT BOOST

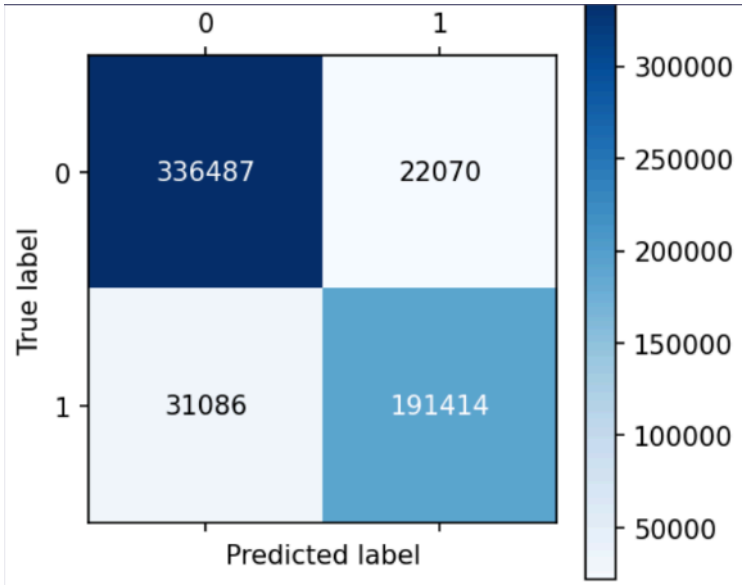

Fig. 6.1 (a) Confusion Matrix

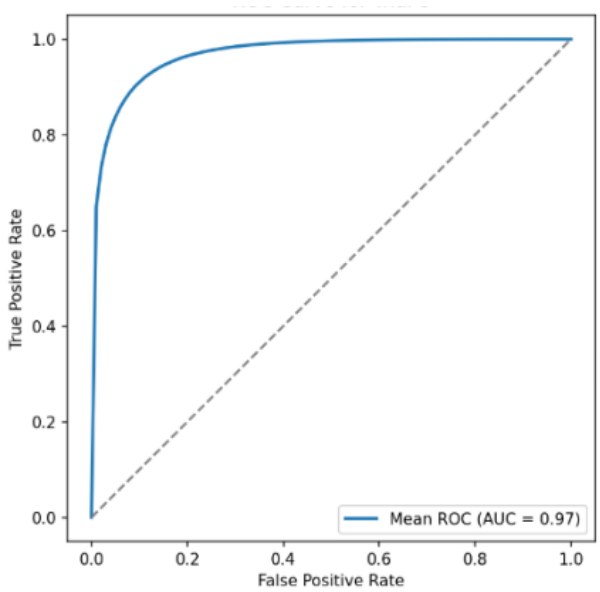

Fig. 6.1 (b) ROC-AUC Curve

MODEL - XG BOOST

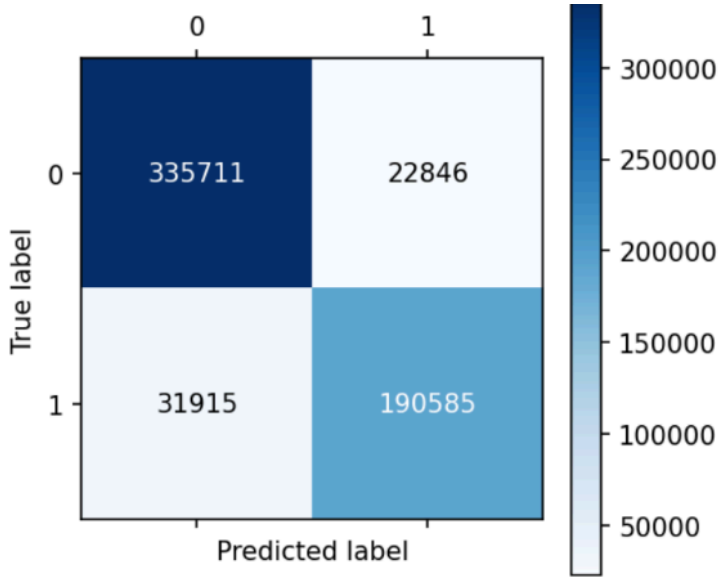

Fig. 6.2 (a) Confusion Matrix

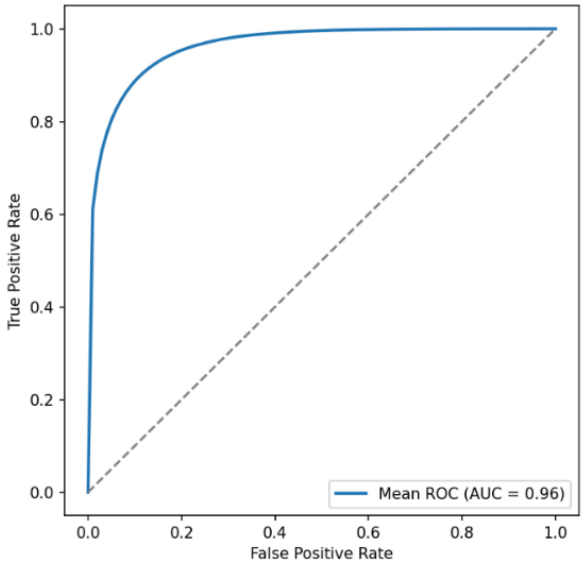

Fig. 6.2 (b) ROC-AUC Curve

MODEL - RANDOM FOREST

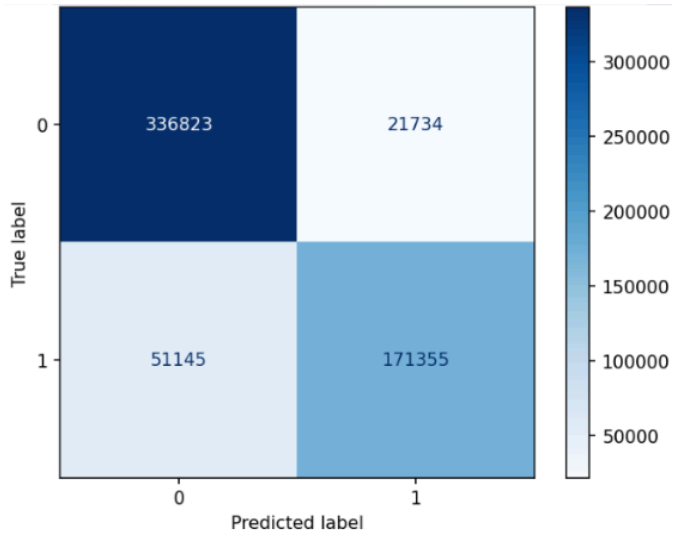

Fig. 6.3 (a) Confusion Matrix

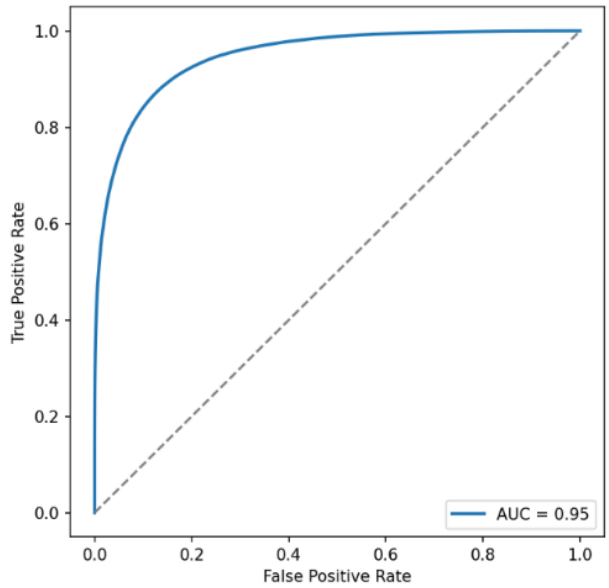

Fig. 6.3 (b) ROC-AUC Curve

MODEL - KNN

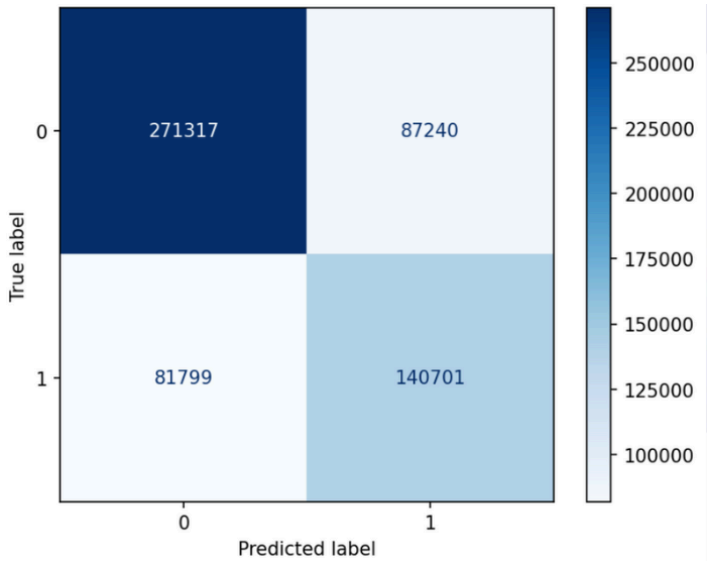

Fig. 6.4 (a) Confusion Matrix

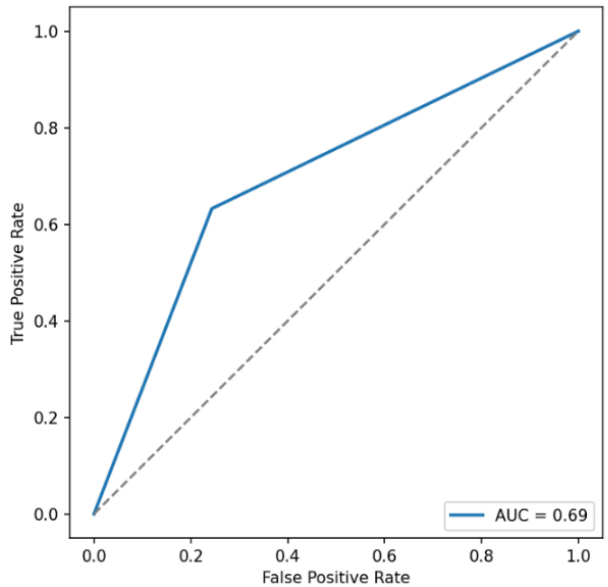

Fig. 6.4 (b) ROC-AUC Curve

7. CONCLUSION AND FUTURE SCOPE

7.1. CONCLUSION

The proposed real-time AI-generated voice detection system demonstrates high effectiveness in identifying synthetic speech, with CatBoost emerging as the top-performing model across all evaluation metrics. The model's low inference time, combined with high precision, recall, and ROC-AUC, highlights its suitability for real-time deployment where speed and accuracy are critical.

Comparative analysis shows that CatBoost and XGBoost significantly outperform traditional models like KNN and Random Forest, especially in terms of precision, robustness, and responsiveness. While KNN's high latency limits its practical use, models like CatBoost strike an ideal balance between computational efficiency and detection accuracy.

These findings underscore the importance of optimized machine learning techniques for fast and reliable detection of AI-synthesized voices. The study supports the deployment of lightweight, high-performing models in security-critical applications such as voice authentication, media verification, and telecommunication fraud prevention.

7.2. FUTURE SCOPE

While the current model demonstrates strong performance in detecting AI-generated voices, real-world deployment requires further enhancements to ensure robustness, efficiency, and scalability. As deepfake voice technologies continue to evolve, it becomes essential to adapt and extend detection systems for broader and more practical applications.

Future work will focus on deploying the model on resource-constrained edge devices, such as smartphones and IoT systems, enabling real-time detection in environments like voice assistants, mobile authentication apps, and smart surveillance. This will support broader adoption of deepfake detection in everyday use cases where latency and compute limitations are critical.

We also aim to improve feature extraction and generalization by incorporating neural vocoder artifacts and multi-modal input streams. Expanding the dataset to include diverse audio sources—across languages, accents, and synthesis techniques—will enhance the model's adaptability and resilience against evolving generative methods.

8. REFERENCES AND BIBLIOGRAPHY

[1] Tanveer, M. U., Munir, K., Amjad, M., Rehman, A. U., & Bermak, A. (2024). Unmasking the Fake: Machine Learning Approach for Deepfake Voice Detection. IEEE Access

[2] Bird, J. J., & Lotfi, A. (2023). Real-time detection of ai-generated speech for deepfake voice conversion. arXiv preprint arXiv:2308.12734.

[3] Ren, H., Lin, L., Liu, C. H., Wang, X., & Hu, S. (2025, April). Improving generalization for ai-synthesized voice detection. In Proceedings of the AAAI Conference on Artificial Intelligence (Vol. 39, No. 19, pp. 20165-20173).

[4] Lim, S. Y., Chae, D. K., & Lee, S. C. (2022). Detecting deepfake voice using explainable deep learning techniques. Applied Sciences, 12(8), 3926.

[5] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. Advances in neural information processing systems, 31.

[6] Paul, S., Das, P., Kashem, A., & Islam, N. (2024). Sustainable of rice husk ash concrete compressive strength prediction utilizing artificial intelligence techniques. Asian Journal of Civil Engineering, 25(2), 1349-1364.

[7] Chen, T., & Guestrin, C. (2016, August). Xgboost: A scalable tree boosting system. In Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining (pp. 785-794).

[8] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. Annals of statistics, 1189-1232.

[9] Wang, Y., Pan, Z., Zheng, J., Qian, L., & Li, M. (2019). A hybrid ensemble method for pulsar candidate classification. Astrophysics and Space Science, 364, 1-13.

[10] Breiman, L. (2001). Random forests. Machine learning, 45, 5-32.

[11] Breiman, L. (1996). Bagging predictors. Machine learning, 24, 123-140.

[12] Verikas, A., Vaiciukynas, E., Gelzinis, A., Parker, J., & Olsson, M. C. (2016). Electromyographic patterns during golf swing: Activation sequence profiling and prediction of shot effectiveness. Sensors, 16(4), 592.

[13] Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. IEEE transactions on information theory, 13(1), 21-27.

[14] Deep-Voice DF:
https://www.kaggle.com/datasets/birdy654/deep-voice-deepfake-voice-recognition

[15] Yamagishi, J., Todisco, M., Sahidullah, M., Delgado, H., Wang, X., Evans, N., ... & Nautsch, A. (2019). Asvspoof 2019: Automatic speaker verification spoofing and countermeasures challenge evaluation plan. ASV Spoof, 13.

[16] Liu, X., Wang, X., Sahidullah, M., Patino, J., Delgado, H., Kinnunen, T., ... & Lee, K. A. (2023). Asvspoof 2021: Towards spoofed and deepfake speech detection in the wild. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 31, 2507-2522.

[17] Reimao, R., & Tzerpos, V. (2019, October). For: A dataset for synthetic speech detection. In 2019 International Conference on Speech Technology and Human-Computer Dialogue (SpeD) (pp. 1-10). IEEE.

[18] Mitrović, D., Zeppelzauer, M., & Breiteneder, C. (2010). Features for content-based audio retrieval. In Advances in computers (Vol. 78, pp. 71-150). Elsevier.

[19] Wang, Y. X., & Zhang, Y. J. (2012). Nonnegative matrix factorization: A comprehensive review. IEEE Transactions on knowledge and data engineering, 25(6), 1336-1353.

[20] Ontivero-Ortega, M., Lage-Castellanos, A., Valente, G., Goebel, R., & Valdes-Sosa, M. (2017). Fast Gaussian Naïve Bayes for searchlight classification analysis. Neuroimage, 163, 471-479.

[21] Dalianis, Hercules. (2018). Evaluation Metrics and Evaluation.
10.1007/978-3-319-78503-5_6.

[22] Chicco, D., & Jurman, G. (2020). The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. BMC genomics, 21, 1-13.

[23] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019, July). Optuna: A next-generation hyperparameter optimization framework. In Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining (pp. 2623-2631).